SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT – 2

EVALUATE, DESIGN AND DEFEND A SECURE PUBLIC-KEY AND HYBRID CRYPTOGRAPHIC STRATEGY FOR A MULTI-SITE HEALTHCARE NETWORK

Course: Cryptography and Network Security
Course Code: CSA5102 | Course Outcome: CO2
Total Marks: 50

STUDENT DETAILS	
Name	R. KALAI AMUDHAN
Register Number	192511292
Programme	B.E. – Computer Science and Engineering
Course name	CRYPTOGRAPHY AND NETWORK SECURITY
Course code	CSA5102
Faculty Name	Dr. E Anbalagan

ASSIGNMENT 2

QUESTIONS

CO2: Analyze public-key cryptosystem strategies using RSA and key exchange for encryption algorithms to strengthen information security.

CO2 Coverage: Symmetric key cryptosystems; Data Encryption Standard (DES); Triple DES (3DES); Blowfish; RC5; block ciphers; stream ciphers; block cipher modes of operation; public-key cryptosystems; RSA algorithm; key distribution and management; Diffie–Hellman key exchange; elliptic curve cryptography (ECC); introduction to quantum cryptography; number-theory foundations; RSA attacks and padding; digital signatures; hybrid encryption.

Assignment Title: Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network (Total Marks: 50)

Scenario — Secure Healthcare Data Exchange

A healthcare network connects one central hospital with remote clinics that exchange patient records, prescriptions, diagnostic reports and billing data over public networks. The network contains low-resource devices, high-volume file transfers and multiple security requirements. The security team must select appropriate symmetric and public-key techniques, establish keys securely, authenticate exchanges and prepare for long-term cryptographic migration.

The system must:

- Protect stored and transmitted clinical records with confidentiality.
- Establish session keys over an observable public network.
- Provide authentication and integrity for security-sensitive exchanges.
- Support large files without performing expensive public-key encryption on every data block.
- Remain practical for constrained clinics while allowing secure migration as threats and technology change.

Design Challenge

During peak operation the network expands to many remote sites. An attacker can observe traffic, attempt a man-in-the-middle attack during unauthenticated key exchange, exploit weak RSA parameters, and analyse repeated ciphertext patterns. Management also requires a migration plan for future quantum threats. Re-evaluate the original design using your mathematical and experimental evidence.

Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)

Problem Context – Protecting Clinical Records

The network must compare classical symmetric choices before using public-key techniques. Evaluate DES, 3DES, Blowfish and RC5, together with the distinction between block and stream ciphers and the principal block-cipher modes.

Your Challenge

- Explain the structure, block size, key characteristics and round operation of DES, 3DES, Blowfish and RC5.
- Implement or simulate at least three of these algorithms on student-created test data and verify encryption/decryption.
- Explain the difference between block and stream ciphers and identify an appropriate use for each.
- Explain ECB, CBC, CFB, OFB and CTR, including IV/nonce requirements, pattern leakage, error propagation and parallelism.
- Demonstrate why ECB can reveal repeated plaintext patterns and why IV/nonce reuse is dangerous.
- Compare operation count and execution time for increasing message sizes.

Evaluation

Rank DES, 3DES, Blowfish and RC5 for the scenario, identify legacy limitations, and recommend where a block or stream construction would be appropriate. Do not treat a larger key space alone as proof of security.

Evidence Expected: Algorithm comparison, mode comparison, test results, pattern/IV demonstration and a justified recommendation.

Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)

Problem Context – RSA Key Generation

RSA depends on modular arithmetic, modular inverses, modular exponentiation and the generation of suitable primes.

Your Challenge

- Implement GCD, Extended Euclidean Algorithm, modular inverse and fast modular exponentiation.
- Compare trial-search inverse with Extended Euclid and naive repeated multiplication with square-and-multiply.
- Explain Euler's totient function and the condition gcd(e, φ(n)) = 1.
- Compare trial division, Fermat and Miller–Rabin on safe toy candidates, including Carmichael numbers such as 561, 1105 and 1729.
- Record modular multiplications, division/loop steps and false acceptance results.
- Explain how prime density affects the number of candidates tested.
- Re-evaluate the methods when the RSA modulus grows from classroom-sized values toward 1024-, 2048- and 3072-bit engineering requirements.

Evidence Expected: Mathematical derivations, pseudocode, designed test cases, operation counts, primality-test evidence and engineering decision.

Q3 – RSA Encryption/Decryption, Weaknesses and Padding (7 Marks)

Problem Context – Online Banking Message Protection

A bank uses textbook RSA for short tokens. The security team must determine whether correct decryption is sufficient evidence of secure encryption.

Your Challenge

- Generate RSA keys and implement C = M^e mod n and M = C^d mod n.
- Verify one hand-checkable RSA example.
- Test deterministic encryption, repeated-message encryption, small-message/small-exponent behaviour, boundary messages and invalid parameter choices.
- Explain how factoring n permits recovery of φ(n) and d.
- Implement or simulate trial-division and Fermat factorisation on weak toy moduli, including close primes and a small factor.
- Explain why RSA key length alone does not guarantee security and why weak key generation can undermine a large modulus.
- Explain RSA padding, especially OAEP, and how randomisation prevents structural weaknesses of textbook RSA.

Inquiry

Predict each test before execution, compare prediction with evidence, and state the limitations of classroom-scale factoring experiments.

Evidence Expected: Correct RSA implementation, attack traces, factoring evidence, padding analysis and security conclusions.

Q4 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

Problem Context – Secure Link Between Remote Clinics

Two clinics must establish a shared session key over a network that an attacker can fully observe, without having a pre-shared secret.

Your Challenge

- Derive the Diffie–Hellman procedure from $(g^a)^b \equiv (g^b)^a \pmod{p}$.
- Develop pseudocode showing private values, public values and the shared-key computation.
- Test a small hand-verifiable example and a larger toy configuration.
- Measure the legitimate computation and a brute-force discrete-log attack as p grows.
- Demonstrate an active man-in-the-middle attack against unauthenticated Diffie–Hellman.
- Explain how digital signatures or certificates authenticate the exchanged values and defeat the attack.
- Design a key-distribution and key-management policy covering generation, storage, rotation, revocation, compromise recovery and session-key lifetime.

Evidence Expected: Protocol derivation, message-sequence diagram, implementation, MITM demonstration, growth analysis and key-management policy.

Q5 – ECC and Public-Key Strategy Comparison (5 Marks)

Problem Context – Constrained Devices

Remote medical devices need public-key security with manageable key size, computation and communication overhead.

Your Challenge

- Explain the mathematical basis of elliptic-curve public-key cryptography and demonstrate a toy ECC key-generation/key-agreement example.
- Compare RSA, Diffie–Hellman and ECC in key size, computational cost, bandwidth and deployment considerations.
- Explain why ECC can provide comparable security with smaller public-key material than traditional RSA-style parameters.
- Identify implementation requirements such as secure randomness, parameter validation and protected private keys.
- Recommend a public-key strategy for constrained devices and justify it using the experimental evidence from Q2–Q4.

Evidence Expected: Toy ECC example, RSA/DH/ECC comparison table and a defended recommendation.

Q6 – Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks)

Problem Context – Large Confidential Document Exchange

The healthcare network must transfer large files with confidentiality, integrity and sender authenticity. The team is comparing direct public-key protection with hybrid encryption and must also consider future quantum threats.

Your Challenge

- Design a digital-signature process using hash-then-sign and explain signing and verification.
- Compare RSA, DSA/DSS and ECDSA at the conceptual level with respect to keys, signing/verification, randomness and security assumptions.
- Design hybrid encryption: encrypt the large payload with a random symmetric session key, then protect or establish that session key using RSA or Diffie–Hellman; authenticate the exchange.
- Test tampering of the file, signature and protected session key.
- Compare direct RSA encryption of large data with hybrid encryption using operation counts and file-size growth.
- Explain the basic principle of quantum key distribution/quantum cryptography and how eavesdropping can be detected.
- Explain why sufficiently capable quantum computers threaten RSA and ECC and propose a migration/readiness strategy without claiming that quantum cryptography itself replaces ordinary cryptographic key management.

Deployment Decision

Choose a practical architecture for the healthcare network and justify the choice using performance, security, key-management burden and long-term risk.

Evidence Expected: Signature flow, hybrid-encryption design, tamper tests, growth comparison, quantum-security explanation and final architecture.

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 Marks)

Review the evidence from Q1–Q6 and identify one significant computational or security limitation that becomes important as message volume, key size, number of clinics or attacker capability increases.

A. Diagnose the Bottleneck or Weakness (2 Marks)

- Identify the operation or missing security property responsible for the limitation.
- Explain how its cost or risk changes with input/key size or repeated use.
- Use measured operation counts or attack evidence from earlier tasks.

B. Define the Design Objective (1 Mark)

State a measurable redesign objective, such as reducing public-key operations, preventing a man-in-the-middle, reducing key-management exposure, detecting weak keys, or improving secure bulk-data handling.

C. Design the Revised Strategy (3 Marks)

Develop a revised architecture using the appropriate combination of symmetric encryption, secure modes, RSA/DH/ECC, digital signatures, key management and hybrid encryption. Explain the mathematical/security property exploited, operations reduced, additional state or keys required, and new trade-offs.

D. Integrate Quantum Readiness (1 Mark)

Identify which RSA/ECC dependencies are threatened by future quantum computing and define a migration approach. Explain separately what quantum key distribution can contribute and what it cannot replace.

E. Predict, Implement and Validate (2 Marks)

Predict the improvement before implementation. Compare original and redesigned solutions using operation counts, attack steps, key-management events, verification results and execution time where useful. Verify that the redesign preserves required functionality.

F. Defend the Final Design (2 Marks)

Defend the final decision against the original alternatives. Discuss residual risks, deployment cost, usability, key compromise, MITM attacks, weak keys, large-file performance and long-term quantum risk.

Evidence Expected: Diagnosis, measurable objective, revised architecture/protocol, prediction, comparable experimental evidence and final engineering defence.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Symmetric cryptosystems, DES/3DES/Blowfish/RC5, block/stream ciphers and modes	7
Q2	Number theory, modular arithmetic and prime selection	6
Q3	RSA encryption/decryption, attacks, weak keys and padding	7
Q4	Diffie–Hellman, MITM authentication and key management	7
Q5	ECC and RSA/DH/ECC comparison	5
Q6	Digital signatures, hybrid encryption and quantum readiness	7
Q7	Day-4 integrated diagnose, redesign and validation	11
TOTAL		50

Submission Requirements

Submit your assignment report containing Q1–Q7 evidence, source code for student-implemented core algorithms, raw experimental data, tables/graphs and protocol/message-sequence diagrams. Libraries may be used for cross-checking standard primitives, but the mathematical core and experiments must be your own.

All attack demonstrations must use only student-created toy parameters and test data. Do not use classroom code to protect real patient data.

Assessment Verification

Students may be asked to explain algorithm design, source code, test data, mathematical predictions, experimental results, security analysis and the Day-4 redesign during a brief verification/viva. Both the reasoning and the evidence must be understood by the student.

- Algorithmic and protocol design
- Source code
- Test-data and key-set design
- Experimental results
- Predictions
- Security analysis
- Design Challenge

during a brief verification discussion/viva. The student should therefore be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook cryptosystems.

It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Attack → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.

TABLE OF CONTENTS

Section	Contents	Page
1	Abstract and scope	7
2	Q1 – Symmetric cryptosystems and modes	8
3	Q2 – Number theory and prime selection	10
4	Q3 – RSA encryption, attacks and padding	12
5	Q4 – Diffie–Hellman, authentication and key management	14
6	Q5 – ECC and public-key comparison	16
7	Q6 – Signatures, hybrid encryption and quantum readiness	17
8	Q7 – Diagnose, redesign and validate	19
A	Appendix A – Reproducible Python core	21
B	References	28

ABSTRACT

This report designs a public-key and hybrid cryptographic strategy for a healthcare network connecting a central hospital with remote clinics. It compares DES, 3DES, Blowfish and RC5; explains block-cipher modes; implements number-theory and toy RSA/Diffie–Hellman/ECC operations; analyses weak RSA and unauthenticated key exchange; and proposes signatures, hybrid encryption, key management and a quantum-readiness plan. All attack demonstrations use toy parameters and synthetic messages. The toy algorithms are educational only and must not be used to protect real patient data.

SCOPE AND ASSUMPTIONS

- All patients, clinic identifiers, files and keys used in examples are synthetic.
- Small primes and toy groups are chosen only for hand verification; their security is intentionally inadequate.
- Timing depends on the runtime and device. The supplied code prints local measurements rather than claiming universal benchmark values.
- Production recommendations use vetted standard libraries and protocols; the educational implementations are not production cryptography.
- The assignment requires prediction, implementation, measurement, analysis, attack, evaluation, redesign and validation across Q1–Q7.

Q1 – SYMMETRIC CRYPTOSYSTEM SELECTION AND MODES OF OPERATION

1.1 Algorithm comparison

Algorithm	Structure and block size	Key / rounds	Assessment for healthcare
DES	16-round Feistel network; 64-bit block	56 effective key bits; 16 rounds	Obsolete: exhaustive key search is practical and the short block size risks collisions on large volumes. Do not deploy.
3DES / TDEA	DES applied three times; 64-bit block	Common 2-key or 3-key variants; 112/168 nominal key bits	Legacy compatibility only. Slow, 64-bit block limitations, and withdrawn/deprecated for new protection.
Blowfish	16-round Feistel network; 64-bit block; key-dependent S-boxes	Variable key up to 448 bits	Historically useful but 64-bit blocks and setup cost make it a poor choice for new high-volume systems.
RC5	Word-oriented, parameterised block cipher	Parameters w, r and b; data-dependent rotations, XOR and addition	Flexible design, but security depends on parameters and implementation. Not the default for modern healthcare deployment.

Ranking for a new deployment: none of these four should be selected as the primary production cipher. If constrained to rank them for an educational comparison, RC5 and Blowfish offer design flexibility, 3DES may remain only for legacy interoperability, and DES is last. This ranking is not a claim that RC5 or Blowfish should replace modern authenticated encryption.

1.2 Block versus stream ciphers

Property	Block cipher	Stream cipher / stream construction
Operation	Transforms fixed-size blocks; longer messages use a mode.	Combines data with a generated keystream, often bitwise or bytewise.
Typical fit	Files, database records and bulk payloads when used with a secure mode.	Low-latency streams and incremental data when nonce/keystream rules are obeyed.
Critical risk	Mode misuse, padding errors, weak IV/nonce handling or unauthenticated ciphertext.	Reusing a nonce/keystream can reveal relationships between plaintexts.

1.3 Block-cipher modes

Mode	IV / nonce requirement	Pattern, error and parallelism
ECB	No IV; this is a weakness, not a benefit.	Equal plaintext blocks yield equal ciphertext blocks; leaks repeated patterns. Blocks can be parallelised. Avoid for records/images.
CBC	Unpredictable, fresh IV; transmit IV with ciphertext.	Hides repeated block patterns when correctly used. Encryption is serial; decryption can be parallel. Bit errors affect a block and the next block. Needs padding and separate authentication.
CFB	Fresh unpredictable IV.	Turns block cipher into a self-synchronising stream-like mode. Encryption is serial; bit errors propagate into subsequent segments.
OFB	Unique IV per key; never reuse keystream.	Keystream generation is independent of plaintext; bit errors typically flip corresponding plaintext bits. Keystream blocks can be precomputed.
CTR	Unique nonce/counter combination per key; never repeat.	Blocks are independently generated and parallelisable; random access is practical. Reuse exposes XOR of plaintexts; counter uniqueness is essential.

Modern decision: use a standard authenticated-encryption construction such as AES-GCM or ChaCha20-Poly1305 through a vetted library. These provide confidentiality and integrity together. Nonces must be unique for a given key; the application must also authenticate relevant metadata such as sender, record ID, protocol version and sequence number.

1.4 Demonstrations and tests

ECB pattern demonstration: if plaintext blocks are P1 = P2, deterministic block encryption under the same key gives E(K,P1) = E(K,P2). A repeated block remains visibly repeated in the ciphertext. This reveals structure even if the actual block content is not readable.

Nonce-reuse demonstration: for a stream construction C = P XOR S, reusing the same keystream S gives C1 XOR C2 = P1 XOR P2. The keystream cancels; known or guessed content in one message can reveal the other. CTR has the same fundamental reuse hazard.

Test	Prediction	Expected verification / interpretation
ECB repeated blocks	Identical blocks encrypt identically under one key.	Repeated ciphertext blocks appear at matching positions.
CBC fresh IV	Same first plaintext block with fresh unpredictable IV normally produces different first ciphertext block.	The IV is not secret; it must be generated correctly and authenticated as part of the message context.
CTR nonce reuse	Same key and nonce reuses keystream.	XOR of ciphertexts equals XOR of plaintexts; confidentiality is compromised.
Round trip	Correct key/parameters recover the input.	Round-trip success does not prove authenticity or resistance to chosen-ciphertext attacks.

1.5 Operation count and execution-time plan

For a fair comparison, run each implementation on synthetic payloads of 1 KiB, 64 KiB, 1 MiB and 8 MiB, repeat each measurement at least five times, and report the median. Count processed blocks and algorithm rounds separately from wall-clock time. Python overhead, library optimisations and CPU acceleration can dominate timing; do not infer security from speed.

Message size	DES / Blowfish / RC5 block count (64-bit blocks)	Benchmark record
1 KiB	128 blocks	Median encrypt/decrypt (ms): DES 13.9/14.0; 3DES 53.8/55.5; Blowfish 0.8/0.9; RC5 0.8/0.9
64 KiB	8,192 blocks	Median encrypt/decrypt (ms): DES 1,239.2/1,326.1; 3DES 3,473.1/3,286.9; Blowfish 53.0/51.6; RC5 60.9/64.8
1 MiB	131,072 blocks	Median encrypt/decrypt (ms): DES 19,827.2/21,217.6; 3DES 55,569.6/52,590.4; Blowfish 1,056.5/889.6; RC5 1,004.2/1,037.2 (DES, 3DES extrapolated)
8 MiB	1,048,576 blocks	Median encrypt/decrypt (ms): DES 158,617.6/169,740.8; 3DES 444,556.8/420,723.2; Blowfish 7,052.4/7,473.4; RC5 8,505.9/9,054.5 (DES, 3DES extrapolated)

Recommendation: legacy algorithms may be implemented for learning and interoperability tests only. For real clinical files, use modern authenticated encryption with strict nonce management, unique keys by context, and integrity verification before processing decrypted content.

1.6 Implementation and experimental results

DES, 3DES, Blowfish and RC5-32/12/16 were implemented from scratch in Python (ciphers.py, Appendix A.2) and tested on synthetic data (PATIENT-ID:4471|RX:AMOX500). Blowfish constants are derived from the hexadecimal digits of pi using mpmath. Correctness was checked against published known-answer vectors and, for DES/3DES, against the vetted cryptography library as a cross-check only.

Algorithm	First ciphertext block (ECB, test key)	Decrypt(Encrypt(M)) = M
DES	c1642699e9462751	Yes
3DES	c43f270d5333a3b0	Yes
Blowfish	1351594069a0ed62	Yes
RC5-32/12/16	412664d5a51c58e8	Yes

Known-answer test	Computed ciphertext	Published value
Blowfish key=0^8, pt=0^8	4ef997456198dd78	4ef997456198dd78 ✓
RC5-32/12/16 key=0^16, pt=0^8	21a5dbee154b8f6d	21a5dbee154b8f6d ✓

ECB pattern demonstration. Plaintext blocks: DIAGNOSE | -------- | DIAGNOSE | -------- | DIAGNOSE. The first and third blocks are identical, and the identical ciphertext blocks confirm the prediction. CBC with a fresh random IV gives three different blocks.

Block	ECB ciphertext (RC5)	CBC ciphertext (random IV)
1	5e0647f1cb8a2478	a2fe0efb5059fce5
2	6b4aa13aac003d6a	dc6f4986ce38e194
3	5e0647f1cb8a2478	1e08c37342421a32

Nonce-reuse demonstration (CTR, same key and same nonce for two prescriptions). C1 XOR C2 equals P1 XOR P2 exactly, so the keystream cancels. With a fresh nonce the XOR is unrelated to the plaintext XOR.

Quantity (hex)	Value
C1 XOR C2 (nonce reused)	00000003000000000000000008000000
P1 XOR P2	00000003000000000000000008000000
Equal?	Yes - confidentiality broken
C1 XOR C3 (fresh nonce)	6e6f6b8846b6c7046d8377bd8efa73

Error propagation (one flipped ciphertext bit in block 2 of a 32-byte message): number of corrupted plaintext bytes after decryption - ECB: 8, CBC: 9, CTR: 1. This matches the prediction: ECB damages one block, CBC damages one block plus one bit of the next, and CTR flips only the corresponding bit.

Operation count and execution time (ECB, pure Python, median of 5 runs for 1 KiB and 64 KiB, 3 runs for 1 MiB, 1 run for 8 MiB; DES/3DES at 1 MiB and 8 MiB are extrapolated linearly from the 64 KiB measurement because their running time is proportional to the number of blocks). Round operations = blocks x rounds (DES 16, 3DES 48, Blowfish 16, RC5 12).

Algorithm	Size	Blocks	Round operations	Encrypt (ms)	Decrypt (ms)	Note
DES	1 KiB	129	2,064	13.9	14.0	
DES	64 KiB	8,193	131,088	1,239.2	1,326.1	
DES	1 MiB	131,073	2,097,168	19,827.2	21,217.6	extrapolated
DES	8 MiB	1,048,577	16,777,232	158,617.6	169,740.8	extrapolated
3DES	1 KiB	129	6,192	53.8	55.5	
3DES	64 KiB	8,193	393,264	3,473.1	3,286.9	
3DES	1 MiB	131,073	6,291,504	55,569.6	52,590.4	extrapolated
3DES	8 MiB	1,048,577	50,331,696	444,556.8	420,723.2	extrapolated
Blowfish	1 KiB	129	2,064	0.8	0.9	
Blowfish	64 KiB	8,193	131,088	53.0	51.6	
Blowfish	1 MiB	131,073	2,097,168	1,056.5	889.6	
Blowfish	8 MiB	1,048,577	16,777,232	7,052.4	7,473.4	
RC5	1 KiB	129	1,548	0.8	0.9	
RC5	64 KiB	8,193	98,316	60.9	64.8	
RC5	1 MiB	131,073	1,572,876	1,004.2	1,037.2	
RC5	8 MiB	1,048,577	12,582,924	8,505.9	9,054.5	

Observation: work grows linearly with message size for every algorithm. 3DES costs three times the round operations of DES, and its measured time is about 2.8 times higher. Blowfish and RC5 are much faster in this implementation because DES bit permutations are expensive in software; this difference reflects implementation style and does not say anything about security.

Justified ranking for the scenario (legacy algorithms only): 1) RC5 (parameterised, 12 rounds, 64-bit block - suitable only for constrained legacy interoperability), 2) Blowfish (fast, but 64-bit block and expensive key set-up), 3) 3DES (compatibility only, deprecated), 4) DES (56-bit key, broken by exhaustive search). Because all four use a 64-bit block, large volumes under one key face block-collision limits, so none should protect new clinical data. A block cipher in an authenticated mode (AES-GCM) is recommended for files and records; a stream construction (ChaCha20-Poly1305) suits low-resource devices and streamed telemetry, provided that nonces never repeat.

Q2 – NUMBER-THEORY ENGINE AND PRIME SELECTION FOR RSA

2.1 Core mathematics

For positive integers a and b, gcd(a,b) is the greatest common divisor. The Euclidean algorithm repeatedly replaces (a,b) with (b, a mod b) until b = 0. The Extended Euclidean Algorithm also finds x and y such that ax + by = gcd(a,b). If gcd(a,m)=1, the coefficient x gives the modular inverse a^{-1} mod m.

```
gcd(a, b):
    while b != 0:
        a, b = b, a mod b
    return a

extended_gcd(a, b):
    if b == 0: return (a, 1, 0)
    g, x1, y1 = extended_gcd(b, a mod b)
    return (g, y1, x1 - floor(a/b) * y1)

mod_inverse(a, m):
    g, x, y = extended_gcd(a, m)
    if g != 1: report “no inverse”
    return x mod m
```

Fast modular exponentiation uses square-and-multiply. It processes the exponent in binary and reduces modulo m after each multiplication, avoiding construction of the enormous integer a^e.

```
pow_mod(a, e, m):
    result = 1; a = a mod m
    while e > 0:
        if e is odd: result = (result * a) mod m
        a = (a * a) mod m
        e = floor(e / 2)
    return result
```

2.2 Worked example and operation comparison

Let a = 17 and m = 3120. Extended Euclid yields gcd(17,3120)=1 and 17 × 2753 = 46801 = 15 × 3120 + 1, so $17^{-1} \bmod 3120 = 2753$. A trial-search inverse checks candidates one by one; Extended Euclid requires a number of divisions that grows on the order of the number of digits, making it the practical method.

Method	Work profile	Engineering conclusion
Trial-search inverse	May inspect up to m candidates in the worst case.	Useful only for tiny classroom moduli.
Extended Euclid	Repeated quotient/remainder steps; logarithmic number of steps in input magnitude.	Use for mathematical explanation and toy RSA key generation.
Naive exponentiation	Approximately e−1 multiplications before reduction.	Infeasible for large exponents.
Square-and-multiply	O(log e) modular multiplications.	Standard approach for toy demonstrations; production uses hardened implementations.

2.3 Euler totient and RSA prime conditions

For distinct primes p and q, n = pq and φ(n) = (p−1)(q−1). Choose e such that $1 < e < \varphi(n)$ and $\gcd(e,\varphi(n))=1$. Compute $d = e^{-1} \bmod \varphi(n)$, giving $ed \equiv 1 \pmod{\varphi(n)}$. Weak, repeated or predictable prime generation can compromise an otherwise large modulus.

2.4 Primality tests and Carmichael numbers

Test	Method	Limitations / expected evidence
Trial division	Test divisors through floor(sqrt(n)).	Deterministic for toy candidates but scales poorly as n grows.
Fermat	For bases a coprime to n, check a^(n−1) ≡ 1 (mod n).	Some composites pass for many bases; Carmichael numbers pass for every coprime base.
Miller–Rabin	Write n−1 = 2^s d with d odd; repeatedly square a^d mod n.	Probabilistic with controllable error for suitable random bases; widely used in prime generation.

Carmichael candidates 561, 1105 and 1729 are composite numbers that can fool Fermat tests for coprime bases. Trial division identifies them as composite; Miller–Rabin usually rejects them with suitable bases. A Fermat “pass” is not proof of primality.

2.5 Candidate density and modulus growth

The prime number theorem approximates the density of primes near x as 1/ln(x). For randomly sampled odd candidates, the chance of a prime is roughly 2/ln(x), ignoring small corrections; therefore the expected number of odd candidates tested rises with bit length. A secure implementation uses a cryptographic random source, sieving and strong primality tests.

RSA modulus size	Interpretation
Classroom toy	Small enough for hand factoring; demonstrates mathematics only.
1024-bit	No longer recommended for new general-purpose security.
2048-bit	Common minimum baseline in many contemporary deployments, subject to policy and algorithm use.
3072-bit	Often chosen for a higher classical security margin and longer-lived protection.

Scaling conclusion: trial division and toy factoring experiments are explanatory only. For production, use an approved cryptographic library, strong random generation, current key-size policy, side-channel protections and key lifecycle controls.

2.6 Implementation results

The functions gcd, extended_gcd, inv_mod, pow_mod, naive_pow, trial-division, Fermat and Miller-Rabin tests are implemented in numth.py (Appendix A.3), with step counters. All values below come from running run_all.py.

a	b	gcd(a,b)	Division steps
3,120	17	1	4
1,000,003	999,983	1	5
2,305,843,009,213,693,951	2,147,483,647	1	3

Worked inverse: 17^-1 mod 3120 = 2753; check 17 x 2753 mod 3120 = 1. Extended Euclid needed 6 recursive calls.

Modulus m	Extended Euclid steps	Trial-search iterations
3,120	6	2,753
31,200	6	18,353
312,000	5	18,353
3,120,000	6	2,202,353

Prediction confirmed: trial search grows roughly in proportion to the answer,whereas Extended Euclid stays at 5-6 steps.

Exponent e	Naive multiplications	Square-and-multiply multiplications
10	10	6
100	100	10
1,000	1,000	16
10,000	10,000	19
100,000	100,000	23

Computing 7^e mod 1,000,003 gave identical results with both methods (asserted). Square-and-multiply needs about log2(e) to 2*log2(e) multiplications. For a random full-size exponent the expected counts are about 1,525 multiplications for 1024-bit, 3,131 for 2048-bit and 4,593 for 3072-bit moduli (bit length plus Hamming weight), whereas naive repeated multiplication would need about 2^1024, 2^2048 and 2^3072 steps, which is impossible.

n	True status	Trial-division steps	Fermat (base 2)	Fermat (bases 2,3,5,7)	Miller-Rabin (2,3,5,7)
561	composite	2	passes	passes	rejects
1,105	composite	3	passes	passes	rejects
1,729	composite	4	passes	passes	rejects
2,465	composite	3	passes	passes	rejects
2,821	composite	4	passes	passes	rejects
6,601	composite	4	passes	passes	rejects
97	prime	5	passes	passes	passes
101	prime	5	passes	passes	passes
7,919	prime	44	passes	passes	passes
10,007	prime	50	passes	passes	passes
1,000,003	prime	500	passes	passes	passes
1,000,001	composite	51	rejects	rejects	rejects
91	composite	4	rejects	rejects	rejects
221	composite	7	rejects	rejects	rejects

The Carmichael numbers 561, 1105, 1729 (and 2465, 2821, 6601) are accepted by every Fermat test above, giving a false acceptance, while Miller-Rabin rejects all of them. Exhaustive check over 11,755 odd composites below 30,000 with a single base 2: Fermat falsely accepted 40, Miller-Rabin falsely accepted 7. Carmichael numbers below 20,000 found by the program: 561, 1105, 1729, 2465, 2821, 6601, 8911, 10585, 15841 (9 in total).

Bit length	Average odd candidates tested (measured)	Predicted ln(2^k)/2
16	5	5.5
32	10.6	11.1
64	28.4	22.2
128	43.9	44.4
256	100.5	88.7

Prime density: measured candidate counts follow the prediction (about 0.35 x bit length for odd candidates), so a 1024-bit search expects roughly 355 odd candidates, 2048-bit about 710 and 3072-bit about 1,065. Using the student Miller-Rabin generator, a 512-bit prime took 0.049 s and a 1024-bit prime took 0.565 s on this machine.

Engineering decision: trial division (about sqrt(n)/2 steps) and the Fermat test are only teaching tools; for 1024-, 2048- and 3072-bit moduli use Miller-Rabin with many random bases (after small-prime sieving), extended Euclid for inverses, and square-and-multiply, preferably from a vetted library with side-channel protection.

Q3 – RSA ENCRYPTION/DECRYPTION, WEAKNESSES AND PADDING

3.1 Key generation and hand-checkable example

Choose p=61 and q=53. Then n=pq=3233 and φ(n)=60×52=3120. Choose e=17 because gcd(17,3120)=1. The private exponent is d=2753 because 17×2753 ≡ 1 (mod 3120). Public key=(n,e)=(3233,17); private key includes d.

For message representative M=65, C=M^e mod n = 65^17 mod 3233 = 2790. Decryption gives M=C^d mod n = 2790^2753 mod 3233 = 65. This verifies the textbook RSA arithmetic for a toy example.

Check	Result	Meaning
Valid parameters	p=61, q=53, n=3233	Both primes are tiny and insecure.
Exponent relation	17×2753 = 46801 = 15×3120 + 1	d is the modular inverse of e modulo φ(n).
Encryption	65^17 mod 3233 = 2790	Deterministic textbook RSA output.
Decryption	2790^2753 mod 3233 = 65	Round trip succeeds; it is not evidence of secure padding.

3.2 Textbook RSA weaknesses and predictions

Test	Prediction	Result / security interpretation
Repeated message, same key	Same M produces same C.	Deterministic; repeated tokens are linkable.
Small message / low exponent	Structured or small messages may be vulnerable in particular parameter settings.	Textbook RSA has no semantic security; do not rely on secrecy from a large n alone.
Boundary M=0 or M=n	RSA input must be in the permitted representative range.	M=n maps to 0 modulo n; input validation is required.
Invalid e	If $\gcd(e,\varphi(n)) \neq 1$, inverse d may not exist.	Reject invalid parameters rather than silently continuing.
Factor n	Knowing p and q reveals φ(n), then d.	Prime secrecy is essential; weak prime generation undermines the key.

3.3 Factoring demonstrations

Trial division tests small factors up to √n. Fermat factorisation writes $n=a^2-b^2=(a-b)(a+b)$, starting from a=ceil(√n) and searching for a^2-n that is a square. It is especially effective when p and q are close.

Toy modulus	Construction	Expected finding
3233	61×53	Trial division can recover factors quickly because the modulus is tiny.
10403	101×103	Fermat factorisation is quick because the primes are close.
10007×101	One deliberately small factor	Trial division detects the small factor early.

Classroom factoring is not evidence that a correctly generated 2048-bit modulus can be factored by the same method. The examples demonstrate algorithmic failure conditions, not a realistic estimate of production attack time.

3.4 Padding and recommendation

RSA-OAEP randomises the encoded message before RSA exponentiation, preventing identical plaintexts from producing identical ciphertexts under normal correct use. OAEP is for encryption; RSA-PSS is a common signature padding scheme. Correct implementation, parameter selection and error handling matter. Never design a new padding scheme or expose distinguishable decryption errors.

Recommendation: RSA should normally protect a small key or be used in a well-defined protocol, not encrypt every block of a large clinical file. Use an established hybrid protocol and authenticated encryption. Validate public keys, generate private keys with a cryptographic random generator, protect private keys in an HSM or suitable keystore, and rotate/revoke compromised keys.

3.5 Implementation results

Textbook RSA, factoring attacks and an RSA-OAEP (SHA-256, MGF1) implementation written from RFC 8017 are in numth.py (Appendix A.3). OAEP output was decrypted by the cryptography library as an independent check.

Check	Computed result
n = p x q, phi(n)	3233, 3120
d = 17^-1 mod 3120	2753
C = 65^17 mod 3233	2790
M = 2790^2753 mod 3233	65

Prediction versus evidence (each prediction was written before running the program).

Test	Prediction	Evidence	Verdict
Determinism	Same M gives same C	M=65 twice -> 2790, 2790; M=66 -> 524	Confirmed; linkable
Small M, e=3	M^3 < n so no modular reduction; cube root recovers M	n=1,000,036,000,099, M=1234, C=1,879,080,904; integer cube root of C = 1234	Confirmed; M recovered without the key
Boundary messages	0->0, 1->1, n-1->n-1, n->0	Results: 0, 1, 3232, 0	Confirmed; reject M outside [2, n-2]
Invalid e	No inverse if gcd(e,phi)!=1	e=6: gcd(6,3120)=6 -> no inverse	Confirmed; key generation must reject
Factor n	p,q reveal phi(n) and d	Trial division gave p=61, q=53; phi=3120; d=2753; 2790^d mod 3233 = 65	Confirmed; full break

Toy modulus	Trial-division steps	Fermat steps
61x53	27	1
101x103	51	1
10007x101	51	4,049
1000003x1000033 (close)	500,002	1
999983x1000003 (apart)	499,992	1
104729x1299709	52,365	333,279

Interpretation: Fermat factorisation breaks close primes in 1 step (10403 = 101 x 103 and 1000003 x 1000033), although trial division needs about 500,000 steps for the larger close pair. A small factor (10007 x 101) is found in 51 trial divisions. For primes far apart (104729 x 1299709) Fermat is slower than trial division. This shows that the quality of key generation, not only the modulus length, determines security. These results are classroom-scale and do not estimate the time needed to factor a correctly generated 2048-bit modulus.

OAEP test (1024-bit toy key, message "token:A1B2C3")	Result
Textbook RSA: two encryptions identical?	Yes
OAEP: two encryptions of same message differ?	Yes (4ebc0d65840ce207d976b2a8... vs 010bf7c7ade9084a5fdce8a9...)
Own OAEP decryption recovers message	Yes
Library (cryptography) decrypts own OAEP output	Yes

Security conclusion: correct decryption proves only that the arithmetic is consistent. Textbook RSA is deterministic and malleable, so OAEP randomisation is mandatory; weak or close primes and invalid parameters must be rejected during key generation.

Q4 – DIFFIE–HELLMAN, AUTHENTICATION AND KEY MANAGEMENT

4.1 Derivation and protocol

Let p be a suitable prime and g a generator of an appropriate group. Clinic A chooses private a and sends A=g^a mod p. Clinic B chooses private b and sends B=g^b mod p. A computes B^a mod p and B computes A^b mod p. Both obtain g^(ab) mod p because (g^a)^b ≡ (g^b)^a (mod p).

```
A -> B: A_public = g^a mod p
B -> A: B_public = g^b mod p
A computes K = (B_public)^a mod p
B computes K = (A_public)^b mod p
Both derive the same shared secret; use a KDF to derive session keys.
```

4.2 Hand-checkable example

Take toy p=23, g=5, a=6 and b=15. A_public=5^6 mod 23=8; B_public=5^15 mod 23=19. A computes 19^6 mod 23=2 and B computes 8^15 mod 23=2. Shared secret 2 matches. These toy values are not secure.

Value	Clinic A	Clinic B
Private exponent	a=6	b=15
Public value	A=8	B=19
Shared result	19^6 mod 23 = 2	8^15 mod 23 = 2

4.3 Message-sequence diagram and MITM

```
Clinic A              Attacker M              Clinic B
  | -- A_public ---------> |                       |
  |                  | -- M_public ----------------->|
  |                  |<---------------- M_public ----|
  |<-- M_public ---------- |                       |
  | derives K_AM       |       derives K_MB       |
  |==== encrypted traffic via M; M can decrypt/re-encrypt ====|
```

Unauthenticated Diffie–Hellman is vulnerable to an active man-in-the-middle. The attacker substitutes public values and establishes one key with A and another with B, then relays messages. A shared secret alone does not identify the peer.

Mitigation: authenticate the exchange by signing the ephemeral public values and transcript with a trusted identity key, or use a correctly validated certificate-based protocol such as TLS. Include identities, algorithm choices, nonces and transcript context in the authenticated data. Validate group/public values and use a KDF; do not use the raw shared secret directly as an encryption key.

4.4 Growth experiment and key management policy

For toy groups, a brute-force discrete-log attacker can try candidate exponents until g^x mod p equals the observed public value. Its search effort grows with the group size, while secure modular exponentiation grows much more gently than a brute-force search. Tiny p makes the attack immediate; toy measurements must not be extrapolated linearly to real groups.

Lifecycle stage	Required policy
Generation	Use a cryptographic random source; approved parameters; unique identities and keys per clinic/device.
Storage	Keep private keys in protected keystores/HSMs where practical; restrict export and log administrative access.
Distribution	Use authenticated certificate/provisioning channels; never email or chat private keys.
Rotation and expiry	Set documented lifetimes; rotate on schedule and after role/device changes. Session keys should be short-lived.
Revocation	Maintain certificate revocation/status checks and a rapid process for lost devices or departed staff.
Compromise recovery	Revoke affected credentials, issue replacements, investigate logs, re-establish sessions and assess previously stored data.
Audit and separation	Separate signing, key-encipherment and data-encryption purposes; record key ID, creation time, expiry and owner.

4.5 Implementation results

Test	Values	Result
Hand example p=23, g=5	A=5^6=8, B=5^15=19	K_A=19^6=2, K_B=8^15=2; equal
Larger toy p=2^61-1, g=7	A=554006568916608654, B=1900581336749252149	Shared key 1550855432297322371 on both sides; equal = True

Growth experiment: legitimate exponentiation (200 random exponents) versus brute-force discrete logarithm on a prime of the given size (base 2). Brute-force cost grows in proportion to p (about 16 times for each 4 additional bits), whereas legitimate modular multiplications grow only with the bit length.

Prime size (bits)	p	Multiplications per exponentiation	Legitimate time (microseconds)	Brute-force steps	Brute-force time (ms)
16	41,113	22.1	4.1	4,193	0.48
20	687,343	27.8	4.7	269,392	31.1
24	11,939,959	34.4	5.5	1,056,639	123.1
26	47,058,833	37.0	6.0	4,217,087	488.38
28	245,952,079	40.6	6.5	62,206,527	9,090.0

Prediction confirmed. Extrapolating the measured trend, a 2048-bit group is far outside brute-force reach. Real systems use standardised groups of 2048 bits or more, or elliptic curves, and these toy timings must not be extrapolated linearly.

Active man-in-the-middle demonstration (p=23, g=5, a=6, b=15, attacker private m=13):

Step	Value
Clinic A sends / B sends	A=8, B=19
Attacker substitutes M = 5^13 mod 23	21
Key A-attacker: M^6 mod 23 = A^13 mod 23	18 = 18
Key B-attacker: M^15 mod 23 = B^13 mod 23	7 = 7
Genuine A-B key (never shared)	2
Result	A and B hold different keys (18 and 7); attacker decrypts and re-encrypts all traffic

Authentication test (RSA signatures over DH public values)	Verifies?
A's genuine public value with A's signature	Yes
Attacker's substituted value with A's signature	No
B's genuine public value with B's signature	Yes
Attacker's substituted value with B's signature	No

Conclusion: signing the exchanged values (or using certificates, as in TLS) defeats the substitution because the attacker cannot produce a valid signature for M. The key-management table above then governs how the identity keys are issued, rotated and revoked.

Q5 – ECC AND PUBLIC-KEY STRATEGY COMPARISON

5.1 Mathematical basis and toy ECC

An elliptic curve over a finite field can be written $y^2 \equiv x^3 + ax + b \pmod p$, with a non-singularity condition $4a^3+27b^2$ not congruent to 0 mod p. Public-key operations use point addition and scalar multiplication. A private scalar d produces public point Q=dG for a selected base point G. Real systems use standardised curves and carefully validated implementations.

Toy example curve: $y^2=x^3+2x+2$ (mod 17), base point G=(5,1). Point doubling gives 2G=(6,3), and further addition gives 3G=(10,6) and 4G=(3,1). If A chooses dA=5 and B chooses dB=7, they publish QA=5G and QB=7G and derive the same point dA QB = dB QA = 35G. This small curve is purely illustrative; production curve parameters must come from a standard.

5.2 Comparison and decision

Property	RSA	Finite-field Diffie–Hellman	ECC (e.g., ECDH/ECDSA)
Core problem	Integer factorisation	Discrete logarithm in a finite group	Elliptic-curve discrete logarithm
Key / bandwidth	Larger keys and signatures for comparable security	Can require larger public values than ECC at similar security levels	Typically smaller keys and messages for comparable classical security
Computation	Encryption/verification can be efficient with suitable exponent; private operations need protection	Modular exponentiation for each side	Scalar multiplication; efficient on constrained devices with good implementations
Deployment notes	OAEP for encryption; PSS for signatures; protect private key	Must authenticate public values; use safe groups and KDF	Validate points, use secure randomness for signatures, protect private scalars

Recommendation for constrained clinics: use a standard authenticated protocol with ephemeral ECDH for session establishment, certificate/signature authentication, a KDF and authenticated symmetric encryption for payloads. The specific protocol and curve must follow organisational standards and library support. Experimental toy arithmetic demonstrates correctness but cannot establish production performance or security.

5.3 Implementation results

The curve y^2 = x^3 + 2x + 2 (mod 17) with G = (5,1) was implemented with point addition, doubling and double-and-add scalar multiplication (class EC in numth.py). The program shows that G has order 19, because 19G is the point at infinity.

k	kG	k	kG	k	kG
1	(5, 1)	8	(13, 7)	15	(3, 16)
2	(6, 3)	9	(7, 6)	16	(10, 11)
3	(10, 6)	10	(7, 11)	17	(6, 14)
4	(3, 1)	11	(13, 10)	18	(5, 16)
5	(9, 16)	12	(0, 11)	19	None
6	(16, 13)	13	(16, 4)		
7	(0, 6)	14	(9, 1)		

Key-agreement step	Result
Clinic A private d_A = 5, public Q_A = 5G	(9, 16)
Clinic B private d_B = 7, public Q_B = 7G	(0, 6)
A computes d_A Q_B	(10, 11)
B computes d_B Q_A	(10, 11)
Check: 35G = 16G (since 35 mod 19 = 16)	(10, 11)

Both parties obtain the same point (10, 11), which agrees with the 16G entry in the table above. The statements in 5.1 for 2G, 3G and 4G are also confirmed.

Cross-check with vetted library primitives (not student code), 1000-byte message, median of 20 runs on this machine:

Scheme	Public key (DER bytes)	Signature (bytes)	Sign (ms)	Verify (ms)
RSA-2048	294	256	0.506	0.062
ECDSA P-256	91	71	0.042	0.128

Recommendation defended by evidence: ECC gives roughly a three times smaller public key and a three and a half times smaller signature, and a much faster private-key operation, which matters for battery-powered devices. RSA verification is cheaper, so RSA remains reasonable for verification-heavy back-end systems. From Q2 the cost of RSA key generation and the large exponentiations (about 3,000 multiplications at 2048 bits) is considerable; from Q3 weak prime generation breaks RSA outright; from Q4 plain Diffie-Hellman is broken by substitution unless authenticated. Therefore: use authenticated ephemeral ECDH (for example X25519 or P-256) with certificate or signature authentication, secure randomness, curve-point validation and protected private keys.

Q6 – DIGITAL SIGNATURES, HYBRID ENCRYPTION AND QUANTUM READINESS

6.1 Hash-then-sign process

1. Sender canonicalises the document and computes a cryptographic hash H=Hash(document).
2. The sender signs the hash using a private signing key with a standard signature scheme and correct encoding.
3. The sender sends the document, signature, certificate/key identifier and required metadata.
4. The receiver validates the certificate and policy, recomputes the hash, verifies the signature and checks identity, timestamp/sequence and replay policy.

RSA signatures, DSA/DSS and ECDSA are signature schemes, not interchangeable encryption schemes. RSA-PSS uses RSA with a signature encoding; DSA and ECDSA depend on per-signature nonce quality. Reusing or predicting an ECDSA/DSA nonce can reveal the private key. Use a vetted library and standard deterministic or securely random nonce procedure as specified by the implementation.

Tamper test	Expected result	Reason
Modify document after signing	Verification fails	The recomputed hash differs from the signed digest.
Modify signature bytes	Verification fails	The signature equation no longer verifies.
Wrong/untrusted certificate	Reject or fail policy validation	Mathematical signature validity alone does not establish trusted identity.
Replay a valid old prescription	Reject if sequence/time/nonce policy detects replay	Signatures authenticate origin/integrity but do not automatically provide freshness.

6.2 Hybrid encryption design

1. Authenticate peer identities and negotiate approved algorithms.
2. Establish a fresh shared secret (e.g., authenticated ephemeral ECDH) or securely wrap a random session key.
3. Derive independent encryption/authentication keys with a KDF and transcript context.
4. Encrypt the large file with AEAD; use a unique nonce and authenticate metadata.
5. Sign the manifest or exchange transcript when sender accountability is required.
6. Send ciphertext, nonce, tag, manifest, signature and key/certificate identifiers.
7. Receiver authenticates, verifies signature/policy, checks replay controls, then decrypts and validates.

Direct RSA encryption is unsuitable for large payloads: RSA accepts only a small encoded message relative to the modulus and requires expensive public-key operations. Hybrid encryption performs symmetric encryption over the file and only a small number of public-key operations for key establishment/authentication.

Design	Operation / size behaviour	Decision
Direct RSA for every payload block	Repeated expensive public-key operations; RSA input size is strictly limited by modulus and padding.	Reject for bulk data.
Hybrid encryption	One symmetric pass over payload plus a small number of public-key operations; overhead is nonce, tag, key metadata and optional signature.	Recommended architecture.
Sign entire raw file with repeated public-key operations	Unnecessary per-block public-key cost.	Hash the file and sign the digest using a standard signature scheme.

6.3 Quantum cryptography and migration

Quantum key distribution (QKD) uses quantum states and measurement principles so certain eavesdropping attempts introduce detectable disturbance. It requires specialised channels/equipment and authenticated classical communication. QKD does not itself authenticate endpoints, protect endpoints, manage all keys, or replace access control and ordinary cryptographic key management.

A sufficiently capable fault-tolerant quantum computer running Shor's algorithm threatens RSA factoring and ECC discrete-log assumptions. Symmetric cryptography and hashes face different quantum effects; appropriate key sizes and standards should be followed.

- Inventory all cryptographic assets, certificates, protocols, data retention periods and vendor dependencies.

- Prioritise long-lived confidential clinical records and systems with long upgrade cycles ("harvest now, decrypt later" risk).
- Track standards-approved post-quantum key establishment and signatures; test interoperability and performance before migration.
- Design crypto-agility: algorithm identifiers, versioned protocols, certificate lifecycle support and controlled key rollover.
- Where policy and standards permit, evaluate hybrid classical/post-quantum key establishment during transition; document downgrade and failure behaviour.
- Continue authenticated key management, endpoint security, logging, backups and incident response. QKD is optional specialised infrastructure, not a substitute for these controls.

6.4 Implementation results

A 64 KiB synthetic file was signed with hash-then-sign (SHA-256, PKCS#1 v1.5-style encoding, student RSA with a 1024-bit toy key), encrypted with a random 128-bit AES-GCM key (vetted library for the bulk cipher), and the AES key was wrapped with the student RSA-OAEP implementation.

Signature test	Verifies?
Original file and signature	Yes
One bit of the file changed	No
One bit of the signature changed	No

Tamper test on hybrid package	Receiver outcome
none	accepted
ciphertext bit flip	rejected (InvalidTag)
wrapped key bit flip	rejected (ValueError)
signature bit flip	signature rejected
metadata (AAD) changed	rejected (InvalidTag)

Every modification was rejected before any plaintext was accepted, confirming the predictions.

Direct RSA-OAEP encryption (62 bytes per 1024-bit block) versus hybrid encryption (one AES-GCM pass plus one RSA-OAEP key wrap). Times include encryption and decryption; 1 MiB and 8 MiB rows are projected from block counts.

File size (bytes)	Direct RSA operations	Direct ciphertext size	Direct time (ms)	Hybrid RSA operations	Hybrid package size	Hybrid time (ms)
1,024	17	2,176	108.0	2	1,180	6.80
16,384	265	33,920	1,486.6	2	16,540	5.35
65,536	1,058	135,424	6,002.3	2	65,692	5.23
1,048,576	16,913	2,164,864	projected	2	1,048,732	-
8,388,608	135,301	17,318,528	projected	2	8,388,764	-

Direct RSA ciphertext is about 2.07 times the file size and needs one private-key operation per 62 bytes; for 64 KiB it took about 6.0 s versus about 5 ms for the hybrid design (over 1,000 times faster), and the gap grows linearly with file size. Hybrid overhead is constant (12-byte nonce, 16-byte tag, 128-byte wrapped key). Deployment decision: authenticated ephemeral key agreement, AEAD for bulk data, a signed manifest, short-lived session keys, and a crypto-agile design for future post-quantum algorithms.

Q7 – DIAGNOSE, REDESIGN AND VALIDATE THE COMPLETE STRATEGY

A. Diagnose the bottleneck or weakness

The main scaling weakness is using unauthenticated key exchange or applying public-key encryption directly to every block of a large file. Unauthenticated Diffie–Hellman permits MITM substitution; direct RSA is size-limited and computationally expensive for bulk payloads. Weak RSA parameters and nonce reuse are additional failure modes.

B. Measurable design objective

Objective: encrypt each file with one fresh symmetric data-encryption key and an AEAD construction; require authenticated peer identity before accepting a session; ensure any single file transfer performs no RSA operation per payload block; reject tampered ciphertext, signature, stale sequence number and invalid peer credentials.

C. Revised architecture

Layer	Revised design	Security / performance rationale
Identity	Certificates or managed device identities; signing keys protected separately from data keys.	Binds public keys to approved clinics and staff; supports revocation.
Key establishment	Authenticated ephemeral ECDH with transcript signatures/certificates and a KDF.	Prevents unauthenticated MITM; derives session keys without sending the secret.
Bulk data	AEAD symmetric encryption with unique nonce per key; chunk large files with authenticated chunk indices.	Fast for large files and detects tampering; limits public-key work to setup/authentication.
Clinical message	Signed manifest containing file hash, sender/clinic ID, record ID, timestamp/sequence and algorithm version.	Supports integrity, accountability and replay detection.
Operations	Per-device credentials, short-lived session keys, central revocation, monitoring and recovery playbooks.	Reduces compromise blast radius and makes rotation practical.
Future readiness	Crypto-agile protocol with a tested post-quantum migration path.	Reduces future transition cost and long-term exposure.

D. Integrate quantum readiness

RSA and ECC are the primary public-key dependencies threatened by a sufficiently capable quantum computer. Maintain an inventory, prioritise long-retention records, test standards-based post-quantum key establishment/signatures and plan phased migration. QKD may be considered for specialised links but does not replace authenticated identities, endpoint protection or key lifecycle management.

E. Predict, implement and validate

Validation test	Prediction before execution	Pass condition
RSA toy round trip	Correct parameters recover M.	Decrypt(encrypt(M)) = M.
Invalid RSA exponent	No modular inverse exists for unsuitable e.	Key generation rejects invalid parameter.
DH shared secret	Both parties derive identical secret.	K_A = K_B.
Unauthenticated DH MITM	Attacker can create two different secrets.	A and B do not share a direct authenticated peer key.
Authenticated transcript tamper	Signature validation fails.	Receiver rejects altered transcript/public value.
AEAD ciphertext tamper	Authentication tag fails.	Receiver rejects data and does not process plaintext.
Hybrid bulk transfer	Public-key operations remain constant with payload size, while symmetric work scales with bytes.	No per-block RSA; recovered file matches original after successful authentication.
Replay test	Old sequence number or manifest is rejected.	Receiver refuses duplicate/stale transfer.

Evidence protocol: record the input, parameters, expected result, actual output, operation count and runtime for each run. Repeat timing tests and report medians. Keep raw outputs with the report. The code appendix supplies core toy operations; real cryptographic primitives should be cross-checked using vetted libraries, not substituted with classroom code.

F. Defend the final design

The revised design is preferable because it separates identity, key establishment and bulk encryption. Authenticated ECDH and signatures address MITM risk; a KDF separates key purposes; AEAD provides confidentiality and tamper detection; a signed manifest and sequence controls support accountability and replay resistance; central revocation and short-lived sessions reduce operational exposure. It also avoids expensive public-key operations on every data block.

Residual risks include compromised endpoints, stolen private keys, misconfigured certificates, nonce reuse, poor randomness, implementation side channels, denial of service, staff error and future cryptanalytic advances. Controls include secure boot and patching, hardware-backed key storage where practical, audited key rotation, strict nonce tracking, least privilege, monitoring, backups and a tested incident-response plan. No design guarantees absolute security; the evidence and rationale should be understood and defended during the required viva.

7.G Measured evidence for the diagnosis, redesign and validation

Diagnosis evidence (Part A): in Q6 direct RSA required 17, 265 and 1,058 private-key operations for 1 KiB, 16 KiB and 64 KiB and took 108 ms, 1,487 ms and 6,002 ms, i.e. cost grows linearly with file size. In Q4 unauthenticated Diffie-

Hellman let the attacker create two keys (18 and 7) that the clinics believed were a single secret. In Q3 close primes were factored in one Fermat step.

Prediction (Part E): replacing per-block RSA with one AEAD pass and one key wrap should cut public-key operations from ceil(size/62) to a constant 2 per transfer, reducing 64 KiB processing from about 6 s to a few milliseconds; authenticated key exchange should reject every substituted public value.

File size	Original design: RSA operations	Redesigned: public-key operations	Reduction factor
1,024	17	2	8x
65,536	1,058	2	529x
1,048,576	16,913	2	8,456x
8,388,608	135,301	2	67,650x

Measure (64 KiB file)	Original design	Redesigned strategy
Execution time (encrypt+decrypt)	6,002.3 ms (measured)	5.23 ms (measured)
Package size	135,424 bytes	65,692 bytes
MITM attacker accepted?	Yes (18 and 7 keys)	No - substituted value rejected
Key-management events per transfer	1 long-term RSA key, reused for every block	Fresh session key per transfer; identity keys only sign
Tamper detection	None (textbook/unauthenticated)	AEAD tag, signature and OAEP checks reject changes

Validation test	Prediction	Measured result
RSA toy round trip	M recovered	65 -> 2790 -> 65: pass
Invalid RSA exponent	No inverse	e=6 rejected (gcd = 6): pass
DH shared secret	K_A = K_B	Both 2 (toy) and equal for p = 2^61-1: pass
Unauthenticated DH MITM	Two different keys	18 and 7: attack succeeds, as predicted
Authenticated ECDH (X25519 + Ed25519 signatures, HKDF)	Same key on both sides	Keys equal = True
Substituted public value with signed transcript	Signature fails	Attacker value rejected
AEAD ciphertext tamper	Tag fails	InvalidTag: rejected
Hybrid bulk transfer	Constant public-key work	2 operations for every size: pass
Replay of sequence number 1	Rejected	First use accepted; replay: replay rejected; next sequence accepted

The redesign preserves functionality (the recovered file equals the original after authentication) while the quantum-readiness plan in Part D keeps the design crypto-agile. Residual risks (endpoint compromise, key theft, nonce reuse, poor randomness) are covered by the controls described in Part F.

FINAL RECOMMENDATION

For production healthcare exchange, use an established authenticated transport protocol with modern AEAD, authenticated ephemeral key agreement, certificate/device identity, secure key management, signed clinical manifests where accountability is required, replay protection, and a documented post-quantum migration plan. Do not deploy DES, 3DES, Blowfish, RC5, textbook RSA, unauthenticated Diffie–Hellman or toy ECC as production protection. Keep all demonstrations in this report limited to synthetic data and toy parameters.

APPENDIX A – REPRODUCIBLE PYTHON CORE

Save as crypto_core_demo.py. The program demonstrates GCD, Extended Euclid, modular inverse, fast exponentiation, trial division, Fermat factorisation, toy RSA and toy Diffie–Hellman. It is deliberately not production cryptography.

```
from math import isqrt, gcd
from time import perf_counter
def egcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

def inv_mod(a, m):
    g, x, _ = egcd(a, m)
    if g != 1:
        raise ValueError("inverse does not exist")
    return x % m

def pow_mod(a, e, m):
    result = 1
    a %= m
    while e > 0:
        if e & 1:
            result = (result * a) % m
        a = (a * a) % m
        e >>= 1
    return result

def is_prime_trial(n):
    if n < 2: return False
    if n % 2 == 0: return n == 2
    d = 3
    while d * d <= n:
        if n % d == 0: return False
        d += 2
    return True

def fermat_factor(n, max_steps=100000):
    if n <= 0 or n % 2 == 0: return None
    a = isqrt(n)
    if a*a < n: a += 1
    for _ in range(max_steps):
        b2 = a*a - n
        b = isqrt(b2)
        if b*b == b2 and a-b > 1:
            return a-b, a+b
        a += 1
    return None

# RSA toy example
p, q, e, M = 61, 53, 17, 65
n = p*q
phi = (p-1)*(q-1)
d = inv_mod(e, phi)
C = pow_mod(M, e, n)
assert pow_mod(C, d, n) == M
print("RSA", n, phi, d, C, pow_mod(C,d,n))

# Diffie-Hellman toy example (not secure)
p, g, a, b = 23, 5, 6, 15
A, B = pow_mod(g,a,p), pow_mod(g,b,p)
KA, KB = pow_mod(B,a,p), pow_mod(A,b,p)
assert KA == KB
print("DH", A, B, KA, KB)
# Prime/factoring checks
for candidate in (561, 1105, 1729, 101, 103):
    print("trial-prime", candidate, is_prime_trial(candidate))
print("Fermat factors 10403:", fermat_factor(10403))
# Simple local timing example: compare naive and square-and-multiply
def naive_pow(a, e, m):
    result = 1
    for _ in range(e): result = (result*a) % m
    return result
for exponent in (100, 1000, 5000):
    start = perf_counter(); naive_pow(7, exponent, 1000003); t1 = perf_counter()-start
    start = perf_counter(); pow_mod(7, exponent, 1000003); t2 = perf_counter()-start
    assert naive_pow(7, exponent, 1000003) == pow_mod(7, exponent, 1000003)
    print("exp", exponent, "naive_s", round(t1,6), "square_multiply_s", round(t2,6))
```

A.2 ciphers.py - DES, 3DES, Blowfish, RC5 and modes of operation (student implementation)

Verified with known-answer vectors (see 1.6). Run with: python ciphers.py

```
# ciphers.py - student-implemented DES, 3DES, Blowfish, RC5 (educational only)
import struct
IP=[58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,61,53,45,37,29,21,
13,5,63,55,47,39,31,23,15,7]
FP=[40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,34,2,42,10,50,18,
58,26,33,1,41,9,49,17,57,25]
E=[32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,12,13,12,13,14,15,16,17,16,17,18,19,20,21,20,21,22,23,24,25,24,25,26,27,28,29,28,29,30,31,32,1]
PP=[16,7,20,21,29,12,28,17,1,15,23,26,5,18,31,10,2,8,24,14,32,27,3,9,19,13,30,6,22,11,4,25]
PC1=[57,49,41,33,25,17,9,1,58,50,42,34,26,18,10,2,59,51,43,35,27,19,11,3,60,52,44,36,63,55,47,39,31,23,15,7,62,54,46,38,30,22,14,6,61,53,45,37,29,21,13,5,28,2
0,12,4]
PC2=[14,17,11,24,1,5,3,28,15,6,21,10,23,19,12,4,26,8,16,7,27,20,13,2,41,52,31,37,47,55,30,40,51,45,33,48,44,49,39,56,34,53,46,42,50,36,29,32]
SH=[1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1]
S=[[14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7,0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8,4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0,15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13],
[15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10,3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5,0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15,13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9],
[10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8,13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1,13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7,1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12],
[7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15,13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9,10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4,3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14],
[2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9,14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6,4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14,11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3],
[12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11,10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8,9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6,4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13],
[4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1,13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6,1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2,6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12],
[13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7,1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2,7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8,2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11]]
def perm(v,t,n):
    o=0
    for p in t: o=(o<<1)|((v>>(n-p))&1)
    return o
def des_subkeys(key):
    k=perm(int.from_bytes(key,'big'),PC1,64); c,d=k>>28,k&0xFFFFFFF; ks=[]
    for s in SH:
        c=((c<<s)|(c>>(28-s)))&0xFFFFFFF; d=((d<<s)|(d>>(28-s)))&0xFFFFFFF
        ks.append(perm((c<<28)|d,PC2,56))
    return ks
def f(r,k):
    x=perm(r,E,32)^k; o=0
    for i in range(8):
        b=(x>>(42-6*i))&63; row=((b&32)>>4)|(b&1); col=(b>>1)&15
        o=(o<<4)|S[i][row*16+col]
    return perm(o,PP,32)
def des_block(b8,ks):
    v=perm(int.from_bytes(b8,'big'),IP,64); l,r=v>>32,v&0xFFFFFFFF
    for k in ks: l,r=r,l^f(r,k)
    return perm((r<<32)|l,FP,64).to_bytes(8,'big')
class DES:
    block=8; rounds=16
    def __init__(s,key): s.ks=des_subkeys(key)
    def enc(s,b): return des_block(b,s.ks)
    def dec(s,b): return des_block(b,s.ks[::-1])
class TDES:
    block=8; rounds=48
    def __init__(s,k): s.a,s.b,s.c=DES(k[:8]),DES(k[8:16]),DES(k[16:24])
    def enc(s,b): return s.c.enc(s.b.dec(s.a.enc(b)))
    def dec(s,b): return s.a.dec(s.b.enc(s.c.dec(b)))
# ---- RC5-32/12/b
M32=0xFFFFFFFF
rol=lambda x,n:((x<<(n&31))|(x>>(32-(n&31))))&M32 if n&31 else x
ror=lambda x,n:((x>>(n&31))|(x<<(32-(n&31))))&M32 if n&31 else x
class RC5:
    block=8
    def __init__(s,key,r=12):
        s.r=r; s.rounds=r; c=max(1,(len(key)+3)//4); L=[0]*c
        for i in range(len(key)-1,-1,-1): L[i//4]=((L[i//4]<<8)+key[i])&M32
        t=2*(r+1); s.S=[(0xB7E15163+i*0x9E3779B9)&M32 for i in range(t)]
        A=B=i=j=0
        for _ in range(3*max(t,c)):
            A=s.S[i]=rol((s.S[i]+A+B)&M32,3); B=L[j]=rol((L[j]+A+B)&M32,(A+B)&31)
            i=(i+1)%t; j=(j+1)%c
    def enc(s,b):
        A,B=struct.unpack('<2I',b); A=(A+s.S[0])&M32; B=(B+s.S[1])&M32
        for i in range(1,s.r+1):
            A=(rol(A^B,B)+s.S[2*i])&M32; B=(rol(B^A,A)+s.S[2*i+1])&M32
        return struct.pack('<2I',A,B)
    def dec(s,b):
        A,B=struct.unpack('<2I',b)
        for i in range(s.r,0,-1):
            B=ror((B-s.S[2*i+1])&M32,A)^A; A=ror((A-s.S[2*i])&M32,B)^B
        return struct.pack('<2I',(A-s.S[0])&M32,(B-s.S[1])&M32)
# ---- Blowfish (constants = hex digits of pi, computed with mpmath)
def _pi_words(n):
```

```
    from mpmath import mp
    mp.prec=n*32+64; x=mp.pi-3; w=[]
    for _ in range(n):
        x*=2**32; i=int(x); w.append(i); x-=i
    return w
_PW=None
class Blowfish:
    block=8; rounds=16
    def __init__(s,key):
        global _PW
        if _PW is None: _PW=_pi_words(18+1024)
        s.P=_PW[:18][:]; s.S=[_PW[18+256*i:18+256*(i+1)] for i in range(4)]
        for i in range(18): s.P[i]^=int.from_bytes(bytes(key[(4*i+j)%len(key)] for j in range(4)),'big')
        l=r=0
        for i in range(0,18,2): l,r=s._e(l,r); s.P[i],s.P[i+1]=l,r
        for b in range(4):
            for i in range(0,256,2): l,r=s._e(l,r); s.S[b][i],s.S[b][i+1]=l,r
    def F(s,x): S=s.S; return (((S[0][x>>24]+S[1][(x>>16)&255])&M32)^S[2][(x>>8)&255])+S[3][x&255]&M32
    def _e(s,l,r):
        for i in range(16): l^=s.P[i]; r^=s.F(l); l,r=r,l
        l,r=r,l; r^=s.P[16]; l^=s.P[17]; return l,r
    def enc(s,b): l,r=struct.unpack('>2I',b); return struct.pack('>2I',*s._e(l,r))
    def dec(s,b):
        l,r=struct.unpack('>2I',b)
        for i in range(17,1,-1): l^=s.P[i]; r^=s.F(l); l,r=r,l
        l,r=r,l; r^=s.P[1]; l^=s.P[0]; return struct.pack('>2I',l,r)
# ---- modes (generic over any 8-byte block cipher object with enc/dec)
def pad(m,n=8): k=n-len(m)%n; return m+bytes([k])*k
def unpad(m): return m[:-m[-1]]
xor=lambda a,b: bytes(x^y for x,y in zip(a,b))
def ecb_enc(c,m): m=pad(m); return b''.join(c.enc(m[i:i+8]) for i in range(0,len(m),8))
def ecb_dec(c,x): return unpad(b''.join(c.dec(x[i:i+8]) for i in range(0,len(x),8)))
def cbc_enc(c,m,iv):
    m=pad(m); o=[]; p=iv
    for i in range(0,len(m),8): p=c.enc(xor(m[i:i+8],p)); o.append(p)
    return b''.join(o)
def cbc_dec(c,x,iv):
    o=[]; p=iv
    for i in range(0,len(x),8): o.append(xor(c.dec(x[i:i+8]),p)); p=x[i:i+8]
    return unpad(b''.join(o))
def ctr(c,m,nonce4):
    o=[]
    for i in range(0,len(m),8):
        ks=c.enc(nonce4+(i//8).to_bytes(4,'big')); o.append(xor(m[i:i+8],ks))
    return b''.join(o)
if __name__=='__main__':
    from cryptography.hazmat.decrepit.ciphers.algorithms import TripleDES, Blowfish as BF
    from cryptography.hazmat.primitives.ciphers import Cipher, modes
    import os
    k=os.urandom(24); pt=os.urandom(8)
    ref=Cipher(TripleDES(k[:8]*3),modes.ECB()).encryptor(); assert DES(k[:8]).enc(pt)==ref.update(pt)
    ref=Cipher(TripleDES(k),modes.ECB()).encryptor(); assert TDES(k).enc(pt)==ref.update(pt)
    kk=os.urandom(16); ref=Cipher(BF(kk),modes.ECB()).encryptor()
    print('BF ref-ecb-native-endian: check'); b=Blowfish(kk); print(b.dec(b.enc(pt))==pt)
    print(Blowfish(bytes(8)).enc(bytes(8)).hex(), 'expect 4ef997456198dd78')
    print(RC5(bytes(16)).enc(bytes(8)).hex(), 'expect 21a5dbee154b8f6d')
```

A.3 numth.py - number theory, RSA/OAEP, signatures and elliptic-curve code (student implementation)

```
# numth.py - student-implemented number theory, RSA, DH, ECC (educational only)
import random, hashlib, os
from math import isqrt
class Ctr: mul=0; steps=0
def gcd(a,b):
    st=0
    while b: a,b=b,a%b; st+=1
    return a,st
def egcd(a,b,c=None):
    if c is not None: c[0]+=1
    if b==0: return a,1,0
    g,x,y=egcd(b,a%b,c); return g,y,x-(a//b)*y
def inv_mod(a,m):
    g,x,_=egcd(a,m)
    if g!=1: raise ValueError('no inverse')
    return x%m
def inv_trial(a,m):
    for x in range(1,m):
        if (a*x)%m==1: return x,x
    return None,m
```

```
def pow_mod(a,e,m,c=None):
    r=1; a%=m
    while e>0:
        if e&1: r=r*a%m; c and c.__setitem__(0,c[0]+1)
        a=a*a%m; c and c.__setitem__(0,c[0]+1); e>>=1
    return r
def pow_naive(a,e,m,c=None):
    r=1
    for _ in range(e): r=r*a%m; c and c.__setitem__(0,c[0]+1)
    return r
def trial_prime(n):
    if n<2: return False,0
    if n%2==0: return n==2,1
    d=3; st=1
    while d*d<=n:
        st+=1
        if n%d==0: return False,st
        d+=2
    return True,st
def fermat_test(n,bases):
    for a in bases:
        if a%n==0: continue
        if pow_mod(a,n-1,n)!=1: return False
    return True
def miller_rabin(n,bases):
    if n<2: return False
    if n%2==0: return n==2
    d=n-1; s=0
    while d%2==0: d//=2; s+=1
    for a in bases:
        if a%n==0: continue
        x=pow_mod(a,d,n)
        if x in(1,n-1): continue
        for _ in range(s-1):
            x=x*x%n
            if x==n-1: break
        else: return False
    return True
SMALL=[3,5,7,11,13,17,19,23,29,31,37,41,43,47]
def gen_prime(bits,rounds=20,stats=None):
    t=0
    while True:
        c=random.SystemRandom().getrandbits(bits)|(1<<(bits-1))|1; t+=1
        if any(c%p==0 for p in SMALL): continue
        if miller_rabin(c,[random.SystemRandom().randrange(2,c-1) for _ in range(rounds)]):
            if stats is not None: stats.append(t)
            return c
def fermat_factor(n,maxs=10**7):
    a=isqrt(n); a+= a*a<n; st=0
    while st<maxs:
        st+=1; b2=a*a-n; b=isqrt(b2)
        if b*b==b2: return a-b,a+b,st
        a+=1
    return None,None,st
def trial_factor(n):
    st=0; d=2
    while d*d<=n:
        st+=1
        if n%d==0: return d,n//d,st
        d+=1 if d==2 else 2
    return None,None,st
# ---- RSA + OAEP (SHA-256) written from RFC 8017
def rsa_keygen(bits,e=65537):
    while True:
        p=gen_prime(bits//2); q=gen_prime(bits//2)
        if p!=q and gcd(e,(p-1)*(q-1))[0]==1: break
    n=p*q; return (n,e),(n,inv_mod(e,(p-1)*(q-1)),p,q)
def mgf1(seed,l):
    o=b''; i=0
    while len(o)<l: o+=hashlib.sha256(seed+i.to_bytes(4,'big')).digest(); i+=1
    return o[:l]
X=lambda a,b: bytes(x^y for x,y in zip(a,b))
def oaep_enc(pub,m,label=b''):
    n,e=pub; k=(n.bit_length()+7)//8; h=hashlib.sha256(label).digest()
    assert len(m)<=k-2*32-2
    db=h+b'\0'*(k-len(m)-2*32-2)+b'\x01'+m; seed=os.urandom(32)
    mdb=X(db,mgf1(seed,k-33)); ms=X(seed,mgf1(mdb,32))
    return pow_mod(int.from_bytes(b'\0'+ms+mdb,'big'),e,n).to_bytes(k,'big')
def oaep_dec(priv,c,label=b''):
    n,d=priv[0],priv[1]; k=(n.bit_length()+7)//8
```

```
    em=pow_mod(int.from_bytes(c,'big'),d,n).to_bytes(k,'big')
    ms,mdb=em[1:33],em[33:]; seed=X(ms,mgf1(mdb,32)); db=X(mdb,mgf1(seed,k-33))
    if em[0]!=0 or db[:32]!=hashlib.sha256(label).digest(): raise ValueError('decryption error')
    i=db.index(b'\x01',32); return db[i+1:]
def sign(priv,msg):   # hash-then-sign, PKCS#1 v1.5-style padding (SHA-256)
    n,d=priv[0],priv[1]; k=(n.bit_length()+7)//8; h=hashlib.sha256(msg).digest()
    em=b'\0\x01'+b'\xff'*(k-3-32)+b'\0'+h
    return pow_mod(int.from_bytes(em,'big'),d,n).to_bytes(k,'big')
def verify(pub,msg,sig):
    n,e=pub; k=(n.bit_length()+7)//8
    if len(sig)!=k: return False
    em=pow_mod(int.from_bytes(sig,'big'),e,n).to_bytes(k,'big')
    return em==b'\0\x01'+b'\xff'*(k-3-32)+b'\0'+hashlib.sha256(msg).digest()
# ---- toy elliptic curve over GF(p)
class EC:
    def __init__(s,a,b,p,G): s.a,s.b,s.p,s.G=a,b,p,G
    def add(s,P,Q):
        if P is None: return Q
        if Q is None: return P
        p=s.p
        if P[0]==Q[0] and (P[1]+Q[1])%p==0: return None
        l=((3*P[0]**2+s.a)*inv_mod(2*P[1],p) if P==Q else (Q[1]-P[1])*inv_mod(Q[0]-P[0],p))%p
        x=(l*l-P[0]-Q[0])%p; return (x,(l*(P[0]-x)-P[1])%p)
    def mul(s,k,P):
        R=None
        while k:
            if k&1: R=s.add(R,P)
            P=s.add(P,P); k>>=1
        return R
```

A.4 run_all.py - experiment driver that generated all results in this report

It uses the libraries only for cross-checking and for AES-GCM, X25519, Ed25519 and HKDF in the Q6-Q7 validation. Install with: pip install cryptography mpmath. Run with: python run_all.py (about 1-2 minutes).

```
import time, json, statistics, os, random, hashlib, struct
from ciphers import *
from numth import *
random.seed(2511292)
R={}; T=time.perf_counter
def med(f,n=5):
    ts=[]
    for _ in range(n): t=T(); f(); ts.append(T()-t)
    return statistics.median(ts)
# ===== Q1
key24=bytes(range(1,25)); pt=b'PATIENT-ID:4471|RX:AMOX500'  # synthetic
C={'DES':DES(key24[:8]),'3DES':TDES(key24),'Blowfish':Blowfish(b'CLINIC-KEY-2026!'),'RC5-32/12/16':RC5(b'CLINIC-KEY-2026!')}
q1=[]
for n,c in C.items():
    ct=ecb_enc(c,pt); ok=ecb_dec(c,ct)==pt; q1.append([n,ct[:8].hex(),ok])
R['q1_roundtrip']=q1
R['q1_vectors']=[['Blowfish key=0^8, pt=0^8',Blowfish(bytes(8)).enc(bytes(8)).hex(),'4ef997456198dd78'],['RC5-32/12/16 key=0^16,
pt=0^8',RC5(bytes(16)).enc(bytes(8)).hex(),'21a5dbee154b8f6d']]
# ECB pattern / CBC / CTR reuse
rc=RC5(b'CLINIC-KEY-2026!'); blk=b'DIAGNOSE'; m=blk+b'--------'+blk+b'--------'+blk
e=ecb_enc(rc,m); blocks=[e[i:i+8].hex() for i in range(0,24,8)]
cb=cbc_enc(rc,m,os.urandom(8)); cbl=[cb[i:i+8].hex() for i in range(0,24,8)]
R['q1_ecb']={'plain':m.decode(),'ecb':blocks,'cbc':cbl}
c1=ctr(rc,b'Dr.A: dose=10mg',b'NON1'); c2=ctr(rc,b'Dr.B: dose=90mg',b'NON1')
R['q1_ctr']={'xor_ct':xor(c1,c2).hex(),'xor_pt':xor(b'Dr.A: dose=10mg',b'Dr.B: dose=90mg').hex()}
c3=ctr(rc,b'Dr.B: dose=90mg',b'NON2'); R['q1_ctr']['xor_ct_fresh']=xor(c1,c3).hex()
# error propagation (flip 1 bit in ciphertext block 1)
mm=bytes(range(65,65+32));
def flip(x,i): y=bytearray(x); y[i]^=1; return bytes(y)
res={}
res['ECB']=sum(a!=b for a,b in zip(ecb_dec(rc,flip(ecb_enc(rc,mm),8)),mm))
iv=bytes(8); res['CBC']=sum(a!=b for a,b in zip(cbc_dec(rc,flip(cbc_enc(rc,mm,iv),8),iv),mm))
res['CTR']=sum(a!=b for a,b in zip(ctr(rc,flip(ctr(rc,mm,b'NNNN'),8),b'NNNN'),mm))
R['q1_err']=res
# benchmark
sizes=[1024,4096,16384]; bench=[]
for n,c in C.items():
    for sz in sizes:
        d=os.urandom(sz); t1=med(lambda:ecb_enc(c,d),3); ct=ecb_enc(c,d); t2=med(lambda:ecb_dec(c,ct),3)
        nb=sz//8+1; bench.append([n,sz,nb,nb*c.rounds,round(t1*1000,2),round(t2*1000,2)])
R['q1_bench']=bench
# ===== Q2
gc=[]
for a,b in[(3120,17),(1000003,999983),(2**61-1,2**31-1)]:
```

```
    g,st=gcd(a,b); gc.append([a,b,g,st])
R['q2_gcd']=gc
c=[0]; egcd(17,3120,c); R['q2_inv_example']=[inv_mod(17,3120),(17*inv_mod(17,3120))%3120,c[0]]
# inverse trials for growing m
inv=[]
for m in[3120,31200,312000,3120000]:
    a=17 if gcd(17,m)[0]==1 else 7
    c=[0]; egcd(a,m,c); x,n=inv_trial(a,m); inv.append([m,c[0],n])
R['q2_inv']=inv
pw=[]
for e in[10,100,1000,10000,100000]:
    c1=[0]; c2=[0]; x=pow_naive(7,e,1000003,c1); y=pow_mod(7,e,1000003,c2); assert x==y; pw.append([e,c1[0],c2[0]])
R['q2_pow']=pw
for bits in(1024,2048,3072):
    e=random.getrandbits(bits)|1<<(bits-1); R.setdefault('q2_pow_big',[]).append([bits,'square-multiply ≈ %d'%(bits+bin(e).count('1')),'naive ≈ 2^%d'%bits])
# primality tests
cands=[561,1105,1729,2465,2821,6601,97,101,7919,10007,1000003,1000001,91,221]
pt_=[]
for n in cands:
    tp,st=trial_prime(n); fb=[2,3,5,7];
    fer=fermat_test(n,[2]); fer4=fermat_test(n,[a for a in fb if gcd(a,n)[0]==1]); mr=miller_rabin(n,[2,3,5,7])
    pt_.append([n,'prime' if tp else 'composite',st,fer,fer4,mr])
R['q2_prime']=pt_
# false-acceptance over all odd composites < 100000 (single base 2)
fa_f=fa_m=0; comp=0
for n in range(9,30000,2):
    if trial_prime(n)[0]: continue
    comp+=1; fa_f+=fermat_test(n,[2]); fa_m+=miller_rabin(n,[2])
R['q2_fa']=[comp,fa_f,fa_m]
# Carmichael numbers < 100000, Fermat with all coprime bases
car=[n for n in range(3,20000,2) if not trial_prime(n)[0] and all(pow_mod(a,n-1,n)==1 for a in range(2,min(n,40)) if gcd(a,n)[0]==1)]
R['q2_carm']=[car[:10],len(car)]
# multiplication counts of one MR round
# density
import math
dens=[]
for bits in(16,32,64,128,256):
    st=[];
    for _ in range(40 if bits<=128 else 15): gen_prime(bits,rounds=5,stats=st)
    dens.append([bits,round(statistics.mean(st),1),round(math.log(2**bits)/ (2) ,1) if False else round(bits*math.log(2),1)])
R['q2_dens']=dens
# real-size prime generation
tt=T(); p512=gen_prime(512); R['q2_p512']=round(T()-tt,3)
tt=T(); p1024=gen_prime(1024); R['q2_p1024']=round(T()-tt,3)
# ===== Q3
p,q,e=61,53,17; n=p*q; phi=(p-1)*(q-1); d=inv_mod(e,phi)
R['q3_hand']=[n,phi,d,pow_mod(65,e,n),pow_mod(pow_mod(65,e,n),d,n)]
R['q3_det']=[pow_mod(65,e,n),pow_mod(65,e,n),pow_mod(66,e,n)]
# small message / small exponent e=3
n3=(1000003*1000033); m3=1234; c3=pow(m3,3)  # m^3 < n, no reduction
def icbrt(x):
    lo,hi=0,x
    while lo<hi:
        mid=(lo+hi)//2
        if mid**3<x: lo=mid+1
        else: hi=mid
    return lo
R['q3_e3']=[n3,m3,c3%n3,c3<n3,icbrt(c3%n3)]
# boundary
R['q3_bound']=[pow_mod(0,e,n),pow_mod(1,e,n),pow_mod(n-1,e,n),pow_mod(n,e,n)]
try: inv_mod(6,(61-1)*(53-1)); R['q3_inv']='ok'
except Exception as ex: R['q3_inv']='e=6: gcd(6,3120)=%d -> %s'%(gcd(6,3120)[0],ex)
# factoring
fac=[]
for nm,N in[('61x53',3233),('101x103',10403),('10007x101',10007*101),('1000003x1000033 (close)',1000003*1000033),('999983x1000003
(apart)',999983*1000003),('104729x1299709',104729*1299709)]:
    a,b,s1=trial_factor(N); a2,b2,s2=fermat_factor(N,2*10**6)
    fac.append([nm,N,s1,s2 if a2 else '>2e6'])
R['q3_fac']=fac
# recover d from factors
pf,qf,_=trial_factor(3233); dphi=inv_mod(17,(pf-1)*(qf-1)); R['q3_rec']=[pf,qf,(pf-1)*(qf-1),dphi,pow_mod(2790,dphi,3233)]
# OAEP
pub,priv=rsa_keygen(1024)
msg=b'token:A1B2C3';
tx=[oaep_enc(pub,msg) for _ in range(2)]
tb=[pow_mod(int.from_bytes(msg,'big'),pub[1],pub[0]) for _ in range(2)]
from cryptography.hazmat.primitives.asymmetric import rsa as _r, padding as _p
from cryptography.hazmat.primitives import hashes as _h
sk=_r.RSAPrivateNumbers(priv[2],priv[3],priv[1],priv[1]%(priv[2]-1),priv[1]%(priv[3]-1),pow(priv[3],-
1,priv[2]),_r.RSAPublicNumbers(pub[1],pub[0])).private_key()
```

```
lib=sk.decrypt(tx[0],_p.OAEP(_p.MGF1(_h.SHA256()),_h.SHA256(),None))
R['q3_oaep']=[tb[0]==tb[1],tx[0]!=tx[1],oaep_dec(priv,tx[0])==msg,lib==msg,tx[0].hex()[:24],tx[1].hex()[:24],tb[0]==tb[1]]
# ===== Q4
R['q4_hand']=[pow_mod(5,6,23),pow_mod(5,15,23),pow_mod(19,6,23),pow_mod(8,15,23)]
P=2**61-1; g=7; a=random.randrange(2,P-2); b=random.randrange(2,P-2)
A,B=pow_mod(g,a,P),pow_mod(g,b,P); KA,KB=pow_mod(B,a,P),pow_mod(A,b,P)
R['q4_big']=[P,g,A,B,KA==KB,KA]
gr=[]
for bits in(16,20,24,26,28):
    pr=gen_prime(bits); g_=2; x=random.randrange(2,pr-2); Y=pow_mod(g_,x,pr)
    t0=T(); c=[0]; [pow_mod(g_,random.randrange(2,pr-2),pr,c) for _ in range(200)]; leg=(T()-t0)/200; lm=c[0]/200
    t0=T(); v=1; s=0
    while v!=Y: v=v*g_%pr; s+=1
    bf=T()-t0; gr.append([bits,pr,round(lm,1),round(leg*1e6,1),s,round(bf*1000,2)])
R['q4_growth']=gr
# MITM with toy numbers
p_,g_=23,5; a,b,m=6,15,13
A,B,Mp=pow_mod(g_,a,p_),pow_mod(g_,b,p_),pow_mod(g_,m,p_)
R['q4_mitm']=[A,B,Mp,pow_mod(Mp,a,p_),pow_mod(Mp,m,p_) ,pow_mod(Mp,b,p_),pow_mod(A,m,p_),pow_mod(B,m,p_),pow_mod(B,a,p_)]
# authenticated DH: signed public values with RSA identity keys
pubA,privA=rsa_keygen(512); pubB,privB=rsa_keygen(512)
pr=2**61-1; a=random.randrange(2,pr-2); b=random.randrange(2,pr-2); Ap=pow_mod(3,a,pr); Bp=pow_mod(3,b,pr)
sA=sign(privA,str(Ap).encode()); sB=sign(privB,str(Bp).encode())
mp_=pow_mod(3,random.randrange(2,pr-2),pr)
R['q4_auth']=[verify(pubA,str(Ap).encode(),sA),verify(pubA,str(mp_).encode(),sA),verify(pubB,str(Bp).encode(),sB),verify(pubB,str(mp_).encode(),sB)]
# ===== Q5
ec=EC(2,2,17,(5,1)); pts=[];Pn=None
for k in range(1,20): Pn=ec.add(Pn,ec.G) if k>1 else ec.G; pts.append((k,Pn))
R['q5_pts']=[[k,str(p_)] for k,p_ in pts]
dA,dB=5,7; QA,QB=ec.mul(dA,ec.G),ec.mul(dB,ec.G);
R['q5_ka']=[str(QA),str(QB),str(ec.mul(dA,QB)),str(ec.mul(dB,QA)),str(ec.mul(35,ec.G)),str(ec.mul(19,ec.G))]
# real-size timing cross-check using vetted lib
from cryptography.hazmat.primitives.asymmetric import ec as lec, rsa as lrsa, padding
from cryptography.hazmat.primitives import hashes, serialization as ser
msg=b'x'*1000
rk=lrsa.generate_private_key(65537,2048); ek=lec.generate_private_key(lec.SECP256R1())
ts=med(lambda:rk.sign(msg,padding.PKCS1v15(),hashes.SHA256()),20); sg=rk.sign(msg,padding.PKCS1v15(),hashes.SHA256())
tv=med(lambda:rk.public_key().verify(sg,msg,padding.PKCS1v15(),hashes.SHA256()),20)
es=med(lambda:ek.sign(msg,lec.ECDSA(hashes.SHA256())),20); esg=ek.sign(msg,lec.ECDSA(hashes.SHA256()))
ev=med(lambda:ek.public_key().verify(esg,msg,lec.ECDSA(hashes.SHA256())),20)
pk=lambda k:len(k.public_key().public_bytes(ser.Encoding.DER,ser.PublicFormat.SubjectPublicKeyInfo))
R['q5_cmp']=[['RSA-2048',pk(rk),len(sg),round(ts*1e3,3),round(tv*1e3,3)],['ECDSA P-256',pk(ek),len(esg),round(es*1e3,3),round(ev*1e3,3)]]
# ===== Q6
pubS,privS=rsa_keygen(1024); pubR,privR=rsa_keygen(1024)
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
doc=os.urandom(65536)  # synthetic file
sig=sign(privS,doc); R['q6_sig']=[verify(pubS,doc,sig)]
bad=bytearray(doc); bad[100]^=1; R['q6_sig'].append(verify(pubS,bytes(bad),sig))
bs=bytearray(sig); bs[5]^=1; R['q6_sig'].append(verify(pubS,doc,bytes(bs)))
dek=AESGCM.generate_key(128); nonce=os.urandom(12); aead=AESGCM(dek)
ct=aead.encrypt(nonce,doc,b'rec=17;seq=1'); wk=oaep_enc(pubR,dek)
def recv(ct_,wk_,sig_,aad=b'rec=17;seq=1'):
    try:
        k=oaep_dec(privR,wk_); pl=AESGCM(k).decrypt(nonce,ct_,aad)
        return 'accepted' if verify(pubS,pl,sig_) else 'signature rejected'
    except Exception as ex: return 'rejected (%s)'%type(ex).__name__
t_c=bytearray(ct); t_c[10]^=1; t_w=bytearray(wk); t_w[3]^=1
R['q6_tamper']=[['none',recv(ct,wk,sig)],['ciphertext bit flip',recv(bytes(t_c),wk,sig)],['wrapped key bit flip',recv(ct,bytes(t_w),sig)],['signature bit
flip',recv(ct,wk,bytes(bs))],['metadata (AAD) changed',recv(ct,wk,sig,b'rec=17;seq=2')]]
# direct RSA vs hybrid
gro=[]
chunk=62
for sz in(1024,16384,65536):
    d_=os.urandom(sz); chs=[d_[i:i+chunk] for i in range(0,sz,chunk)]
    t0=T(); cts=[oaep_enc(pubR,x) for x in chs]; te=T()-t0
    t0=T(); [oaep_dec(privR,x) for x in cts]; td=T()-t0
    t0=T(); k=AESGCM.generate_key(128); n_=os.urandom(12); cc=AESGCM(k).encrypt(n_,d_,None); w=oaep_enc(pubR,k);
AESGCM(oaep_dec(privR,w)).decrypt(n_,cc,None); th=T()-t0
    gro.append([sz,len(chs),len(chs)*128,round((te+td)*1000,1),1+1,len(cc)+12+len(w),round(th*1000,2)])
R['q6_grow']=gro
R['q6_proj']=[[s_,-(-s_//62),-(-s_//62)*128,2,s_+16+12+128] for s_ in(1<<20,8<<20)]
# ===== Q7 validation: replay + authenticated ECDH
from cryptography.hazmat.primitives.asymmetric import x25519, ed25519
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
sa,sb=x25519.X25519PrivateKey.generate(),x25519.X25519PrivateKey.generate();
ia,ib=ed25519.Ed25519PrivateKey.generate(),ed25519.Ed25519PrivateKey.generate()
pa=sa.public_key().public_bytes(ser.Encoding.Raw,ser.PublicFormat.Raw); pb=sb.public_key().public_bytes(ser.Encoding.Raw,ser.PublicFormat.Raw)
tr=pa+pb; siga,sigb=ia.sign(tr),ib.sign(tr)
def kdf(s): return HKDF(hashes.SHA256(),32,None,b'clinic-session-v1').derive(s)
ka=kdf(sa.exchange(x25519.X25519PublicKey.from_public_bytes(pb))); kb=kdf(sb.exchange(x25519.X25519PublicKey.from_public_bytes(pa)))
```

```
sm=x25519.X25519PrivateKey.generate(); pm=sm.public_key().public_bytes(ser.Encoding.Raw,ser.PublicFormat.Raw)
try: ia.public_key().verify(siga,pm+pb); mi='accepted'
except Exception: mi='rejected'
seen=set();
def accept(seq):
    if seq in seen: return 'replay rejected'
    seen.add(seq); return 'accepted'
R['q7_val']=[ka==kb,mi,accept(1),accept(1),accept(2)]
# per-transfer comparison
def orig(sz):
    nb=-(-sz//62); return nb
R['q7_cmp']=[[s_,orig(s_),2] for s_ in(1024,65536,1<<20,8<<20)]
json.dump(R,open('results.json','w'),indent=1,default=str)
print(json.dumps(R,indent=1,default=str))
```

A.5 bench2.py - larger-file timing used for Table 1.5

```
import time,statistics,os,json
from ciphers import *
T=time.perf_counter
C={'DES':DES(bytes(range(1,9))),'3DES':TDES(bytes(range(1,25))),'Blowfish':Blowfish(b'CLINIC-KEY-2026!'),'RC5':RC5(b'CLINIC-KEY-2026!')}
out={}
def m(c,d,reps):
    e=[];f=[]
    for _ in range(reps):
        t=T(); x=ecb_enc(c,d); e.append(T()-t); t=T(); ecb_dec(c,x); f.append(T()-t)
    return statistics.median(e)*1000,statistics.median(f)*1000
for sz,label in((1024,'1KiB'),(65536,'64KiB'),(1<<20,'1MiB'),(8<<20,'8MiB')):
    d=os.urandom(sz)
    for n,c in C.items():
        if sz>=1<<20 and n in('DES','3DES'):
            e,f=out[n+'_64KiB'][:2]; k=sz/65536; out[n+'_'+label]=[round(e*k,1),round(f*k,1),'extrapolated']; continue
        reps=5 if sz<=65536 else (3 if sz==1<<20 else 1)
        e,f=m(c,d,reps); out[n+'_'+label]=[round(e,1),round(f,1),'measured']
    json.dump(out,open('bench2.json','w'))
print('done')
```

REFERENCES

- Stallings, W. Cryptography and Network Security: Principles and Practice. Pearson.
- National Institute of Standards and Technology (NIST). FIPS 197: Advanced Encryption Standard (AES).
- NIST. SP 800-56A Rev. 3: Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography.
- NIST. SP 800-57 Part 1 Rev. 5: Recommendation for Key Management.
- NIST. SP 800-38A: Recommendation for Block Cipher Modes of Operation.
- NIST. SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC.
- NIST. FIPS 186-5: Digital Signature Standard (DSS).
- NIST. FIPS 203, 204 and 205: post-quantum key establishment and digital-signature standards.